

실행 및 재현 가이드

로컬 실행 · 테스트 · Docker · 재현 실험 — 평가자가 그대로 따라갈 수 있는 최소 절차

대상 코드: 동결 태그 `v1.0.1-submission` · 전체 문서는 저장소 `README.md` · 배포 배경은 `DEPLOY.md`

1. 요구 환경

Python	3.13 권장(컨테이너 이미지 <code>python:3.13-slim</code> · 측정 환경 3.13.9). 3.11 이상이면 대체로 동작하지만 검증한 버전은 3.13 이다
OS	무관(측정 환경 Windows 11 · 컨테이너 Linux)
필수	Git · pip
선택	Docker Desktop(컨테이너 실행) · gcloud CLI(Cloud Run 배포)
외부 API	OpenAI 또는 Anthropic 키(실제 실행) · Tavily 키(웹 검색, 선택)

2. 설치

```
git clone <repo-url>
cd AI-Service-Planning-Multi-Agent
pip install -r requirements.txt          # 또는 버전 고정: requirements-lock.txt
```

3. 환경변수

```
cp .env.example .env
```

변수	예시 값	설명
LLM_PROVIDER	openai	openai anthropic dummy
USE_DUMMY	0	1 이면 키 없이 전체 파이프라인 완주(더미 모드). 키가 없으면 자동으로 더미가 된다
OPENAI_API_KEY	sk-...	실제 실행에 필요. Anthropic 이면 ANTHROPIC_API_KEY
OPENAI_MODEL	gpt-4o-mini	실측에 쓴 모델 ID
TAVILY_API_KEY	tvly-...	선택 — 비우면 웹 검색을 생략하고 LLM 지식 기반으로 동작(참고자료도 비운다)
WORKFLOW_MODE	serial	parallel 은 Research 이후 독립 분석 4분기를 동시 실행(결과·프롬프트 동일, 순서만 다르다). 실측은 parallel
BUDGET_MAX_*	50 / 1.0 / 600000	실행 1건의 호출·비용(USD)·시간(ms) 상한. 초과 시 강제 중단이 아니라 이후 호출 생략
PUBLIC_MAX_*	0	공개 배포용 상한(기본 전부 비활성). 로컬 동작은 바뀌지 않는다
ENABLE_DEMO_TOOLS	0	1 이면 /admin (의도적 Agent 실패 시연) 활성화. 기본 404
PROJECTS_DB_PATH	(비움)	비우면 data/projects.db

4. 로컬 실행

```
uvicorn app.main:app --reload
# 화면      http://localhost:8000/
# API 문서  http://localhost:8000/docs
# 상태 확인 http://localhost:8000/health  ← dummy_mode 가 false 인지 반드시 확인
```

배포·기동 성공 ≠ 실제 LLM 모드. 키가 없거나 `USE_DUMMY=1` 이면 모든 단계가 정상으로 보이면서 더미 출력이 나온다. 확인은 항상 `/health` 의 `dummy_mode` 필드로 한다.

화면은 입력 / 결과(Agent별) / 최종 기획서 / 이력 4개다. 실행이 끝나면 Markdown · JSON · DOCX · PPTX 를 브라우저에서 내려받을 수 있다.

5. 테스트

```
pytest -q          # 전체 675개 · 실제 LLM 호출 없이 결정적
ruff check .       # 정적 검사
```

실측(2026-07-29): CI 에서 `pytest -q --cov=app --cov-fail-under=90` → **675 passed · 커버리지 96.65%**(64초 · 하한 90% 통과). 테스트는 **실제 LLM 을 호출하지 않는다**.

⚠ 로컬에서 재면 **96.93%** 로 높게 나온다 — `.env` 의 API 키 `LANGFUSE_BASE_URL` 이 있으면 `tracing.py` · `llm.py` 의 env 게이트 분기가 실행되기 때문이다. **재현 가능한 값은 키가 없는 CI 쪽이다**. 또 `WORKFLOW_MODE=parallel` 이면 이력 저장 테스트 1건이 환경 차이로 실패할 수 있어 `serial` 로 실행한다.

6. Docker 실행

```
docker compose up --build      # 기본 USE_DUMMY=1 → 키 없이 기동·완주 확인
python scripts/smoke_test.py   # /health + /run 스모크(호스트 → 컨테이너)
```

실제 키로 돌리려면 `.env` 를 채우거나 `USE_DUMMY=0` 을 오버라이드한다. 이력·산출물은 named volume(`project-data` · `project-outputs`)에 저장된다 — host bind mount 로 바꾸면 비루트(uid 10001) 컨테이너가 쓰지 못해 `/health` 는 통과하고 `/run` 에서만 실패할 수 있다.

7. 재현 실험 (선택)

<code>python run_demo.py</code>	파이프라인 처음~끝 관통 확인
<code>python run_compare.py</code>	단일 프롬프트 vs Multi-Agent 비교 → <code>docs/comparison_result.md</code>
<code>python run_multimodel.py</code>	생성 모델별 비교
<code>python run_parallel_bench.py --topics 3 --reps 1</code>	직렬 vs 병렬 wall time·품질·비용
<code>python run_kosena_check.py</code>	KOSENA 28항목 준수 점검

⚠ `run_compare` · `run_multimodel` 은 실제 LLM 을 여러 번 호출하므로 비용이 발생한다.

8. 주의사항

- 외부 API 키가 필요하다. OpenAI(또는 Anthropic) 키 없이는 실제 결과가 나오지 않고 더미 모드로 동작한다. Tavily 키가 없으면 **웹 검색과 출처 인용이 빠진다** — 이 프로젝트의 핵심 차별점이 근거 수집이므로, 평가 시에는 Tavily 키를 넣는 편이 실제 동작에 가깝다.
- 비용**. 실행 1건당 실측 평균 약 **\$0.022**(범위 \$0.016~0.024 · LLM 호출 약 18회 · 소요 약 166초, N=7 기준). 실행 예산 상한이 기본으로 걸려 있다.
- Cloud Run 에서 이력은 보존되지 않는다**. 이력은 SQLite 파일 (`data/projects.db`)이고 **컨테이너 재시작 시 사라진다**. 공개 배포에서는 산출물을 내려받아 보관하는 것으로 대응했다. ⚠ **GCS FUSE + SQLite 조합은 파일 잠금 문제로 금지**(DB 손상). 외부 DB 전환은 제출 범위에서 제외했다.
- 공개 배포는 **--max-instances=1** 이 필수다. 요청·비용 카운터가 프로세스 메모리라 인스턴스가 늘면 상한이 인스턴스 수만큼 곱해져 방어가 조용히 약해진다.
- 이력 조회에 인증이 없다**. 같은 서버에 접근할 수 있으면 누구나 이력을 볼 수 있어, 입력 화면에서 사전 고지한다. 민감한 아이디어는 공개 배포에 넣지 않는 것을 권한다.
- 같은 파일이 다시 생성되지는 않는다**. LLM 출력은 실행마다 달라지므로 이 폴더의 03~06 과 바이트 단위로 같은 결과를 재현할 수 없다. 그래서 그 실행의 State 를 `06_샘플실행정보_고령자복약관리.json` 으로 함께 제출한다.
- Windows 에서 OneDrive 동기화 폴더 안에 저장소를 두면 git 파일 잠금 오류가 날 수 있다.